

Oasis Intelligence

CS 411: Database Systems — Final Project Reflection
Jenish Patel, Alfred Rizk, Rishabh Ranganathan, Vijwal Akula

1. Changes in Project Direction

The final implementation of Oasis Intelligence did not deviate from the original Stage 1 proposal. The core scope — a full-stack application for exploring global agricultural production and trade data — remained unchanged throughout development. The technology stack (a FastAPI backend, Google Cloud SQL for MySQL, and a ReactJS frontend), the seven-entity relational schema, and the overall feature set were all executed as originally proposed, with functionality layered on top rather than the direction changing.

2. Application Usefulness: Achievements and Shortcomings

Oasis Intelligence succeeded in its core goal of letting a user query and visualize global crop production data without writing SQL by hand. The saved-views feature is the clearest example of this: a user builds a filtered query once — crop, country, production element, and year range — and can revisit it later as a named view with a rendered chart, which is a meaningfully different experience from a static, one-off dashboard. Account creation and login also work end-to-end against real data rather than mocked responses, and the multi-select filter interface (crop/country chips, a dual-thumb year-range slider) makes constructing a query more approachable than raw form fields would. The addition of a linear regression forecast on top of the historical chart also moves the application beyond a pure lookup tool, giving a user a lightweight predictive view of where production is trending rather than only where it has been.

Where the application falls short of full usefulness is in coverage and performance. The schema includes Indicators and TradeIndicators tables that are not surfaced as prominently in the interface as ProductionRecords is — a user specifically interested in trade flows currently has less to work with than one interested in production volumes. On the performance side, each API request opens a fresh database connection rather than drawing from a connection pool, which is acceptable at class-project traffic levels but would not hold up under real concurrent use.

3. Schema and Data Source Changes

The main external data sources remained the same. The application still uses the agricultural production and trade-related datasets imported into the original MySQL schema: Users, SavedViews, Crops, Countries, ProductionRecords, Indicators, and TradeIndicators. These tables continue to support the core filtering, visualization, saved-view, and account functionality.

The only schema addition was an internal application-generated table, SavedViewLogs, which records audit history when users create, update, or delete saved dashboard presets. Unlike Crops, Countries, ProductionRecords, Indicators, and TradeIndicators, SavedViewLogs are not imported from a CSV file. Its rows are created automatically by MySQL triggers on the SavedViews table.

Each SavedViewLogs row stores the affected view ID, user ID, action type, timestamp, and message, allowing the frontend to show recent saved-view activity. The table intentionally does not have a foreign key back to SavedViews so that audit history remains available even after a saved view is deleted. Other than this audit-log table, the original external data sources and core schema remained unchanged.

4. ER Diagram and Table Implementation Changes

No changes were made to the ER diagram or the table structure from the original Stage 1 design. The one implementation-level adjustment made during development was enabling `AUTO_INCREMENT` on surrogate primary key columns (for example, `SavedViews.viewId`) that had not originally been configured with a default value-generation strategy. This did not change the schema's structure — only how surrogate keys are populated. In retrospect, `AUTO_INCREMENT` should have been part of the original DDL for every surrogate key column from the start; manually computing the next ID in application code (used briefly as a workaround before the schema was corrected) is more error-prone and does not hold up under concurrent writes.

5. Functionalities Added or Removed

No functionality originally planned was removed. The following functionality was added beyond the baseline CRUD-and-query requirements:

- Full create/read/delete for saved views (an existing view is deleted and recreated rather than updated in place, for simplicity).
- User authentication — signup and login backed by the `Users` table, with session state held client-side for the duration of a browser session.
- Chart visualization for saved views, rendering matching production records as a line chart with one series per crop–country combination.
- Keyword search across crops and countries, letting a user locate relevant records by name instead of manually building filters from the picker lists.
- A chip-based multi-select interface for choosing crops, countries, and the production element, plus a year-range slider — replacing plain HTML form controls with a more usable filter-building experience.
- A linear regression model applied to each saved view's historical production data, projecting a trend line for near-future years alongside the existing chart so users can see a forecasted trajectory rather than just past values.

These were added because the original CRUD-and-query baseline, while sufficient to meet the assignment's SQL requirements, did not by itself constitute a complete user-facing product. Saved views, authentication, and visualization are what turn the underlying schema into something a user would plausibly return to.

6. How Advanced Database Programs Complement the Application

The advanced SQL work in this project directly powers user-facing features:

- The production-query endpoint joins `ProductionRecords` to `Crops` and `Countries` and applies parameterized `IN` clauses over user-selected crop and country lists, translating a multi-select filter UI directly into a single indexed query instead of looping over individual lookups in application code.
- A CTE-based rewrite of a `UNION` query used earlier in the project's SQL requirements reduced redundant computation across the unioned branches — the same query-plan-level thinking carried over into how later endpoints were written, favoring one well-indexed join over multiple round trips.
- Aggregate queries (`GROUP BY` combined with `JOIN` — for example, ranking crops by row count in `ProductionRecords`, or `SELECT DISTINCT` to populate the element picker) support both application features and ad hoc data-quality checks made during development.

Without these techniques, the frontend would need to fetch raw tables and filter or aggregate client-side, which does not scale and defeats the purpose of using a relational database in the first place.

7. Technical Challenges

Jenish Patel

One of the most instructive bugs I encountered was a silent data-corruption issue in the Crops table, caused by a CSV import that didn't account for quoted fields. The source file, `Crops.csv`, had 224 rows, several of which contained commas inside the crop name itself (for example, “Chick peas, dry”), so those fields were wrapped in double quotes per standard CSV convention. When I loaded the file into Cloud SQL, I didn't explicitly tell the import that fields could be quote-enclosed. The importer treated every comma as a column separator, including the ones inside quotes — so the row for that entry split into extra, misaligned columns, and everything after it in the source file was swallowed into a single oversized `cropName` value for one row.

This was hard to catch precisely because it failed silently: every query against the table ran without error, and the API endpoint returned valid JSON — it just returned far less data than it should have. The table still looked structurally fine (a plausible row count, no crash); it just only actually contained four usable rows, with the fourth row's `cropName` field holding hundreds of characters of raw, unparsed CSV data from the rest of the file. I only found it because the frontend crop picker looked sparse — four chips instead of roughly two hundred — which prompted me to inspect the raw API response instead of assuming the endpoint code itself was broken.

The fix was to re-import with the parser explicitly told that fields could be comma-containing and quote-enclosed (in MySQL's `LOAD DATA INFILE`, this means adding `FIELDS TERMINATED BY ','`, `OPTIONALLY ENCLOSED BY '"'`), and to match the source file's actual line endings (`LINES TERMINATED BY '\r\n'`) rather than assuming Unix-style line breaks.

Advice for a future team: if your source data comes from a CSV that might have commas inside a field — product names, descriptions, addresses, anything free-text — never assume a naive comma-split import worked correctly just because the import command exited without an error. Spot-check row counts and a few actual field values after any bulk import, not only whether the command succeeded. A silent, no-error data-corruption bug is more dangerous than one that crashes loudly, because everything downstream — queries, APIs, UI — will appear to work while quietly operating on bad data.

Rishabh Ranganathan

One technical challenge I worked on was adding a meaningful MySQL trigger feature for our saved dashboard presets. Since users can create, update, and delete saved views, I implemented an audit system that automatically records those actions in a separate `SavedViewLogs` table. The goal was for the database itself to log activity whenever the `SavedViews` table changed, instead of having the frontend manually insert log rows.

The main design issue was deciding whether `SavedViewLogs.viewId` should be a foreign key back to `SavedViews`. At first, that seemed logical because each log entry refers to a saved view. However, this would create a problem when a saved view is deleted, since the original row would no longer exist. To preserve deleted-view history, I intentionally did not add a foreign key from `SavedViewLogs` back to `SavedViews`.

I created three triggers: an insert trigger that logs `CREATE`, an update trigger that logs `UPDATE` only when the title or filters actually change, and a delete trigger that logs `DELETE` using the old row values. This helped me understand how MySQL triggers use `NEW` values for inserted or updated rows and `OLD` values for deleted rows.

I also connected the trigger feature to the application by adding a backend endpoint that reads recent log records and displaying them in the frontend under a “Recent Activity” panel. The important part is that the frontend only reads the audit history; the log entries themselves are generated automatically by MySQL triggers.

Advice for a future team: when adding database triggers, try to connect them to a real user-facing feature rather than leaving them hidden in the backend. In our case, the trigger system made saved presets more transparent by giving users a visible history of when views were created, updated, or deleted. Also, think carefully about how

deletion should affect related records. For audit logging, preserving history can be more important than enforcing a direct foreign key to a row that may no longer exist.

Vijwal Akula

One technical challenge I faced was managing the deployment and cost of our MySQL database on Google Cloud Platform. I was responsible for hosting the shared Cloud SQL instance so that the team could import data, connect the backend, and test the application. I initially used the \$50 educational Google Cloud credit available to me, but the instance had been configured using Enterprise Plus with 8 vCPUs, 64 GB of memory, 100 GB of SSD storage, and an additional data cache. This was far more powerful than our project required and consumed the available credit very quickly.

Once the credit was exhausted, Google Cloud suspended the instance, temporarily preventing the team from accessing the database. This was confusing because the database had not failed due to an SQL or application error. Most Cloud SQL controls, including restart and configuration changes, were disabled while the instance was suspended. I had to inspect the billing account, verify that the project was linked correctly, and determine that the problem was caused by the expired credit rather than a database failure.

To restore the project, I activated Google Cloud's \$300, 90-day free trial and linked it to the project. Once billing was restored, the existing Cloud SQL instance automatically became available again without losing our data. I then reduced the configuration from 8 vCPUs and 64 GB of memory to 1 vCPU and 3.75 GB of memory, kept the instance in a single zone, and disabled the unnecessary data cache. The smaller configuration was still sufficient for our queries, procedures, triggers, and backend requests while greatly reducing costs.

Advice for a future team: estimate cloud costs before creating an instance and begin with the smallest configuration that supports the expected workload. Premium database features and large machines may appear useful, but they can quickly exhaust limited project credits without providing any meaningful benefit for a small class application.

I also implemented a stored procedure that automatically creates a default saved view whenever a new user is added. The main challenge was storing a dynamically selected crop and country IDs inside the existing JSON filters column. I researched how MySQL handles JSON values generated from query results and learned about JSON_OBJECT, JSON_ARRAY, and JSON_ARRAYAGG. Using these functions, I was able to let the DB build the filter structure safely instead of manually assembling a JSON string.

<https://dev.mysql.com/doc/refman/8.0/en/json-creation-functions.html>

https://dev.mysql.com/doc/refman/8.0/en/aggregate-functions.html#function_json-arrayagg

Alfred Rizk

Alfred Rizko

One technical challenge I worked on was building our extra-credit Creative Component: a real-time Statistical Yield Forecasting and Volatility Engine within our analytics dashboard ('AnalyticsView.tsx'). The goal was to give food security analysts predictive insights such as 2030 yield targets and climate risk alerts rather than just displaying static historical database charts.

The main challenge was processing multi-decade time-series data on the fly without causing UI rendering lag when users select multiple crops and countries. To solve this, I implemented a least-squares linear regression model ($y = mx + b$) directly over the query results to project yield trajectories out to the year 2030. Additionally, to highlight supply chain risks caused by droughts or yield instability, I developed a statistical model calculating

the coefficient of variation ($\frac{\text{StdDev}}{\text{Mean}}$) to dynamically trigger 'High Volatility' risk badges when crop variations exceed safety thresholds.

I also connected this forecasting system to our backend transaction API ('POST /api/analytics/snapshot'), executing explicit SQL commands under 'REPEATABLE READ' isolation to ensure regional benchmark snapshots are saved atomically into 'SavedViews' without non-repeatable read anomalies.

Advice for a future team: When building advanced analytics or extra-credit features, combine client-side statistical modeling with robust backend database transactions. Performing trend mathematics in the browser while keeping state management atomic in SQL delivers a fast, responsive UI without compromising database consistency.

8. Other Changes from the Original Proposal

No other significant changes occurred between the original proposal and the final application. The project scope, schema, and technology choices proposed at Stage 1 were carried through to the final implementation without major departures.

9. Future Work (Beyond the Interface)

- Hash passwords instead of storing them in plain text in the Users table.
- Move authentication from a client-side sessionStorage object to a server-issued, tamper-resistant token (e.g., a signed JWT or server-side session).
- Introduce connection pooling (e.g., HikariCP) instead of opening a new database connection per request, for better performance under concurrent load.
- Make fuller use of the Indicators and TradeIndicators tables already in the schema — for example, trade-flow-specific saved views and visualizations, not just production.
- Add input validation hardening and rate limiting on write endpoints (signup, view creation) before any real deployment.
- Build a scheduled data-refresh pipeline to pull updated production and trade figures periodically, instead of relying on a one-time CSV import.

10. Division of Labor and Teamwork

Our team divided the project based on the major layers of the application: database setup, data import/cleaning, backend/API development, frontend integration, advanced database programs, testing, and documentation. Vijwal Rao handled the main database creation and cloud database management work, including setting up and maintaining the MySQL database on Google Cloud. He also worked on the stored procedure requirement and helped ensure the database-side logic was integrated into the final project.

Jenish Patel focused on importing and cleaning the CSV data used by the application. This included debugging data-quality issues, validating that the imported crop and production records were usable, and making sure the real datasets could support the frontend filters and database queries. He also contributed to some backend and frontend work as the application was being connected end-to-end.

Alfred Rizk set up the FastAPI backend and handled much of the backend/API development. His work included creating the routes that connect the React frontend to the MySQL database, supporting retrieval and CRUD operations, and making sure user actions in the interface could correctly query or modify the database.

Rishabh Ranganathan worked on parts of the stored procedure and backend integration, but his main responsibility was the trigger requirement. He implemented the SavedViews audit logging system using MySQL

triggers, added the SavedViewLogs table, connected the trigger-generated logs to a backend endpoint, and helped display the activity history in the frontend.

Overall, the team split responsibilities by component while still coordinating during integration. Since the frontend, backend, database schema, and advanced SQL features all depended on each other, we had to communicate about table structure, endpoint behavior, and testing results throughout the project. The workload was distributed so that each member owned a major part of the system while also helping with debugging and final documentation when needed.